

FreshSelect Meals — Security Audit Report

Date: July 2026

Auditor: Manus AI

Scope: Server-side tRPC procedures, authentication/authorization, IDOR vulnerabilities, privilege escalation, and blind enumeration

Status: All critical/high findings resolved

Executive Summary

A comprehensive security audit of the FreshSelect Meals codebase identified **4 vulnerabilities** across two severity levels. All findings have been remediated, regression tests added, and the fixes are deployed to the Manus-hosted dev environment (checkpoint `81c10378`).

ID	Severity	Category	Status
IDOR-1	High	Insecure Direct Object Reference	Fixed
IDOR-2	High	Insecure Direct Object Reference	Fixed
PRIV-1	High	Privilege Escalation	Fixed
BLIND-DELETE	Medium	Blind Enumeration / IDOR	Fixed

Findings and Fixes

IDOR-1 — Assessors Could Read Sub-Resources for Unassigned Clients

Severity: High

Affected procedures: `tasks.byClient`, `notes.byClient`, `documents.byClient`, `services.byClient`, `stageHistory`, `listReferrerNotes`, `sendReferrerNote`, `listClientEmails`, `sendClientEmail`

Description: Assessors are scoped to only see clients assigned to them in the `assessorList` procedure. However, all sub-resource procedures (`tasks`, `notes`, `documents`, `services`, `stage history`) only required `assessorProcedure` authentication — they did not verify that the requested client was actually assigned to the calling assessor. An authenticated assessor could supply any `submissionId` and read or write data for any client in the system.

Fix: Added an ownership check to every affected procedure. After fetching the submission, the procedure verifies `submission.assessorId === ctx.user.id` before returning data. Non-assessor roles (admin, worker, viewer, super_admin) bypass this check and retain full access.

```
// Added to each affected procedure after fetching the submission:
if (ctx.user.role === "assessor" && submission.assessorId !== ctx.user.id)
  throw new TRPCError({ code: "FORBIDDEN", message: "Not assigned to this"
}
```

Tests added: 9 regression tests in `security-audit6.test.ts`

IDOR-2 — Assessors Could Read/Send Referrer Notes and Client Emails for Unassigned Clients

Severity: High

Affected procedures: `listReferrerNotes`, `sendReferrerNote`, `listClientEmails`, `sendClientEmail`

Description: The referrer messaging and client email procedures were protected by `assessorProcedure` but did not scope access to the assessor's assigned clients. An assessor could enumerate referrer conversations and send emails on behalf of any client in the system.

Fix: Same ownership check pattern as IDOR-1 applied to all four procedures. The assessor's `userId` is compared against `submission.assessorId` before any data is returned or action is taken.

Tests added: Covered under IDOR-1 test suite (shared ownership check logic)

PRIV-1 — `updateStaff` Could Modify Another Super Admin Account

Severity: High

Affected procedure: `updateStaff`

Description: The `updateStaff` procedure was protected by `superAdminProcedure`, meaning only super admins could call it. However, there was no check preventing one super admin from modifying another super admin's account. This created a password takeover vector: a super admin could change another super admin's password hash, effectively locking them out or hijacking their account.

Fix: Added an explicit guard at the top of the procedure that throws `FORBIDDEN` if the target user is a `super_admin` and their `id` differs from the caller's `id`.

```
if (targetUser.role === "super_admin" && targetUser.id !== ctx.user.id) {  
  throw new TRPCError({ code: "FORBIDDEN", message: "Cannot modify another user" });  
}
```

Additionally, the Zod input schema for `updateStaff` only accepts `admin | worker | viewer | assessor` as valid role values — `super_admin` cannot be assigned via this procedure at all.

Tests added: 7 regression tests in `security-audit6.test.ts`

BLIND-DELETE — `deleteReferrerNote` Had No Existence Check

Severity: Medium

Affected procedure: `deleteReferrerNote`

Description: The `deleteReferrerNote` procedure accepted a `messageId` and called `DELETE WHERE id = ?` without first verifying the message existed. This allowed an authenticated user to silently probe for valid message IDs (blind IDOR enumeration) — a `DELETE` on a non-existent row returns success, so the caller could infer ID existence from side effects (e.g., whether a subsequent `listReferrerNotes` count changed).

Fix: Added a `getReferrerMessageById` helper to `db.ts` and called it before deletion. If the message does not exist, the procedure throws `NOT_FOUND`.

```
// db.ts — new helper  
export async function getReferrerMessageById(id: number) {  
  const [msg] = await db.select().from(referrerMessages).where(eq(referrerMessageId, id));  
  return msg ?? null;  
}  
  
// routers.ts — deleteReferrerNote  
const msg = await getReferrerMessageById(input.messageId);  
if (!msg) throw new TRPCError({ code: "NOT_FOUND" });
```

Tests added: 2 regression tests in `security-audit6.test.ts`

Additional Security Measures (Pre-Existing)

The following security controls were verified to be correctly implemented prior to this audit:

Control	Implementation
Rate limiting	<code>globalLimiter</code> (100 req/15 min), <code>loginLimiter</code> (10 req/15 min), <code>passwordResetLimiter</code> , <code>referrerCodeLimiter</code>
Content Security Policy	Helmet CSP with strict directives
CORS	Allowlist of known origins only
Session tokens	JWT with <code>HttpOnly; Secure; SameSite=Strict</code> cookies
Password hashing	bcrypt with cost factor 12
Password reset tokens	32 random bytes → SHA-256 hash stored in DB; 1-hour expiry
Invite tokens	32 random bytes → SHA-256 hash stored in DB; 24-hour expiry
SQL injection	Drizzle ORM parameterized queries throughout
Role-based guards	<code>protectedProcedure</code> , <code>staffProcedure</code> , <code>editProcedure</code> , <code>deleteProcedure</code> , <code>adminProcedure</code> , <code>superAdminProcedure</code> , <code>assessorProcedure</code>
Sensitive columns	<code>passwordHash</code> , <code>passwordResetToken</code> excluded from <code>listStaffUsers</code> response via <code>SAFE_USER_COLUMNS</code>

Test Coverage

Test File	Tests	Purpose
security-audit6.test.ts	26	IDOR-1, IDOR-2, PRIV-1, BLIND-DELETE regression tests
assessor-assignment.test.ts	7	Assessor assign/unassign null-safety
password-reset.test.ts	34	Password reset token security, expiry, origin allowlist
All other test files	240	Feature correctness
Total	307/308	1 pre-existing Resend email config failure (unrelated to security)

Recommendations

1. **Monitor assessor access logs** — now that IDOR-¹/₂ are fixed, any `FORBIDDEN` errors from assessor procedures should be logged and alerted on as potential probing attempts.
2. **Consider adding a `super_admin` audit log** — track all `updateStaff` calls with actor/target/fields-changed for compliance.
3. **Rotate the JWT secret** if there is any suspicion that the current secret has been exposed.
4. **Fix the Resend `RESEND_FROM_EMAIL` environment variable** in Vercel — the value stored uses Unicode-escaped angle brackets (`\u003c` / `\u003e`). The `getFromEmail()` helper decodes these at runtime, but the root cause should be corrected in the Vercel dashboard.